2. *allowed_domains*: This is the base address of the URLs that the spider is allowed to crawl.
3. *Start_requests()*: The spider begins to crawl on the requests returned by this method. It is called when the spider is opened for scraping.
4. *Parse():* This handles the responses downloaded for each request made. It is responsible for processing the response and returning scraped data. In the above code, the parse method will be used to save the *response.body* into the HTML file.

Crawling

Crawling is basically following links and crawling around websites. With Scrapy, we can crawl on any website using a spider with the following command:

```
scrapy crawl myFirstSpider
```

Extraction with selectors and items

Selectors: A certain part of HTML Source can be scraped using selectors, which is achieved using CSS or Xpath expressions.

Xpath is a language for selecting nodes in XML documents as well as with HTML, whereas CSS selectors are used to define selectors for associate styles. Include the following code to our previous spider code to select the title of the Web page:

```
def parse(self, response):
    url=response.url
 for select in response.xpath('//title'):
            title=select.xpath('text()').extract()
            self.log("title here %s" %title)
```

Items: Items are used to collect the scraped data. They are regular Python dicts. Before using an item we need to define the item *Fields* in our project's *items.py* file. Add the following lines to it:

```
title=item.Field()
url=item.Field()
```

Our code will look like what's shown in Figure 1.

Figure 1: First item

After the changes in the item are done, we need to make some changes in our spider. Add the following lines so that it can yield the item data:

```
from scrapy_first.items import ScrapyFirstItem

def parse(self, response):
        item=ScrapyFirstItem()
        item['url']=response.url
        for select in response.xpath('//title'):
                title=select.xpath('text()').extract()
                self.log("title here %s" %title)
                item['title']=title
                yield item
```

Now run the spider and our output will look like what's shown in Figure 2.

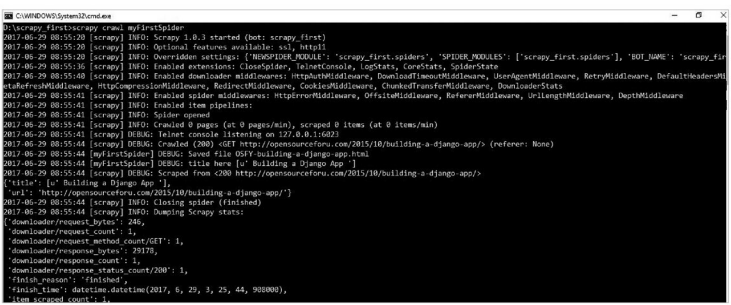

Figure 2: Execution of the spider